기민수

보안, 인프라, AI 보안 엔지니어

만든 프로젝트와 운영하면서 고친 문제를 정리했습니다.

프로젝트

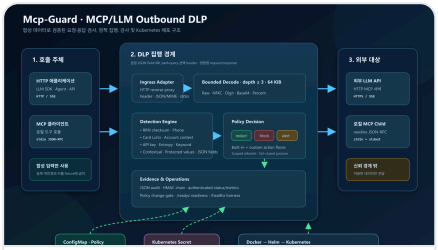
ETK-Scanner

ETK-Scanner

비용 상한이 있는 LLM 소스코드 취약점 진단

2026.07

Python ast, Claude Haiku, Sonnet



Mcp-Guard

LLM과 MCP 통신의 개인정보 유출 방지 프록시

2026.07

Python, FastAPI, uvicorn, httpx, regex

EvidScope

EvidScope

AI 에이전트의 위험 행동을 판정 모델로 가려내 감사 증거로 남기는 보안 도구

2026

JEV-9B, Qwen3-4B, Python, PyTorch, Transformers, PEFT

Docker 컨테이너 보안 가이드 저술

Docker 컨테이너 보안 가이드 저술

위험도와 구현 비용을 기준으로 Docker 컨테이너 보안 대응을 다시 정리한...

2026년 개인 저술

TryHackMe 모의 침투 실습과 보고서

TryHackMe 모의 침투 실습과 보고서

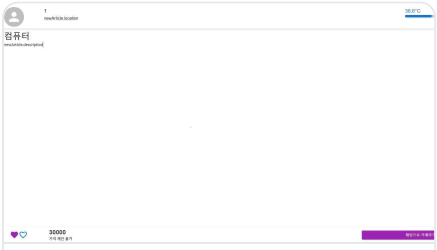
허가된 워게임 환경에서 모의 침투를 연습하고, 공격 경로와 개선 방안을...

WIKI

WIKI

게시판과 권한별 접근 제한, 실시간 채팅을 갖춘 위키 서비스를 프론트엔드부터...

2023년 개인 프로젝트 TypeScript



Carrot 클론

상품을 올리고 둘러보는 거래 서비스를 본떠 화면과 서버를 연결해 본 2022년...

2022년



Kiosk

2021년 9월부터 2022년 1월까지 주문 키오스크의 상품 선택부터 주문 전송까...

2021.09–2022.01

트러블슈팅

고객사 웹 서버 침해사고 두 건 대응과 AWS 재구축

KISA 침해 통보와 서비스 중단 신고로 시작된 두 사고에 현장에서 대응하고, 무결성을 확인한 데이터만 AWS로 옮겨 서버를 다시 세웠습니다.

서버: AWS EC2와 RDS로 재구축

대형병원 키오스크 CMS의 권한 결함 수정과 보안점검 대응

대형병원 키오스크를 원격으로 관리하는 CMS를 혼자 만들어 운영했고, 세션 검증보다 권한 부여가 먼저 일어나던 결함을 고쳐 병원 보안점검을 통과했습니다.

권한 부여 순서: 요청, 세션 검증, 권한 부여

서버 7대 통합과 여러 겹의 백업 설계

노후 서버 7대를 Hyper-V로 합치며 외부에서 바로 들어오던 원격 접속을 없애고, 스냅샷 하나에 기대던 복구를 여러 겹의 백업으로 바꿨습니다.

복구 수단: 변경 제한 백업, AWS 서버 이미지, MSSQL 복제

5성급 호텔 서버의 디스크 고갈 장애와 재발 방지

배포 때마다 쌓인 Docker 이미지와 중지된 컨테이너가 디스크를 채워 서비스가 멈춘 장애를 복구하고, 주기 정리를 자동화해 같은 원인의 장애를 막았습니다.

디스크 정리: cron으로 주기 실행

프로젝트에서 고친 문제

ETK-Scanner	큰 저장소에서 검사 조각이 불어나 비용이 커짐
ETK-Scanner	저비용 분류가 음성이라 하면 상위 검증이 막힘
ETK-Scanner	재현된 결함과 보고할 만한 결함이 섞임
ETK-Scanner	이미 알려진 취약점을 제보 직전까지 감
Mcp-Guard	HTTP만 보는 프록시는 stdio MCP 서버에 닿지 못함
Mcp-Guard	띄어 쓴 번호, 전각 숫자, Base64로 감싼 값이 통과
Mcp-Guard	실제 기밀값이 설정 이력에 남을 위험
Mcp-Guard	원문을 그대로 보내는 경보 설정으로 정책이 약해질 위험
EvidScope	모델이 경보 이름만 보고 답하는 지름길
EvidScope	학습과 평가 사이의 누수
EvidScope	요청마다 서명 원장 전체를 다시 검증하던 병목
Carrot 클론	API 연동 문제 해결
Kiosk	통신 문제 해결

트러블슈팅

고객사 웹 서버 침해사고 두 건 대응과 AWS 재구축

KISA 침해 통보와 서비스 중단 신고로 시작된 두 사고에 현장에서 대응하고, 무결성을 확인한 데이터만 AWS로 옮겨 서버를 다시 세웠습니다.

시기	2023.04–2025.03
환경	IIS 로그, MSSQL, AWS EC2, RDS, 보안그룹

문제 상황

한 고객사의 웹 서버는 오래된 ASP.NET 위에서 돌고 있었고, 이 서버에서 침해사고가 두 번 일어났습니다. 두 사고 모두 제가 현장에 나가 대응했습니다.

1. **통보와 신고** 1차는 KISA의 침해 통보, 2차는 서비스가 멈췄다는 고객 신고로 시작했습니다
2. **접근 되찾기** 시스템 접근과 관리자 계정을 되찾아 로그와 프로세스를 볼 수 있게 했습니다
3. **원인 추적** IIS 로그를 따라가 응용프로그램 풀 안의 악성 실행을 찾았습니다
4. **데이터 분리** MSSQL 데이터를 격리된 PC로 옮겨 무결성을 확인했습니다
5. **다시 세우기** AWS EC2와 RDS에 새로 세우고 운영 IP만 열었습니다

원인 분석

1차 사고는 KISA가 고객사 서버의 침해를 통보해 알게 됐습니다.

2차 사고는 서비스가 멈췄다는 고객 신고로 시작됐습니다. 서버의 CPU와 메모리가 비정상적으로 점유돼 있었고 공격자가 설치한 가상화 환경이 있어, 서버 자원을 몰래 쓰는 크립토마이닝을 의심했습니다.

무엇이 남아 있는지 보려면 시스템 접근과 관리자 계정부터 되찾아야 했습니다. 이 단계 없이는 로그도 프로세스도 확인할 수 없었기 때문입니다.

고객에게 조치가 필요한 이유를 설명할 때는 기존 .NET Framework 버전과 업데이트 버전의 공개 CVE 수, 사고 빈도를 근거로 삼았습니다.

해결 과정

- 현장에서 시스템 접근과 관리자 계정을 되찾고, IIS 로그를 따라가 응용프로그램 풀 안의 악성 실행을 루트킷으로 판단했습니다.
- 구형 미들웨어와 ASP.NET 관련 취약점을 점검하고 조치했습니다.
- MSSQL 데이터는 내부망의 격리된 PC로 옮겨 무결성을 확인한 뒤 분리했습니다.
- 2차 사고 복구 중 MSSQL 백업 복원이 멈춘 원인을 찾아, 업로드 검증 없이 저장된 과대 크기의 악성 이미지 파일을 분리하고 제거했습니다.
- 웹 애플리케이션을 AWS EC2에 새로 세우고 DB를 RDS로 옮겼습니다. 보안그룹은 운영 IP 허용목록만 열고 RDP 접근을 막았습니다.

백업 복원이 중간에 멈춤

해결 멈춘 지점을 따라가 업로드 파일을 점검

결과 검증 없이 저장된 과대 크기 악성 이미지를 분리하고 복원을 마침

VPN을 쓸 수 없는 구간

해결 서비스 포트와 출발지 IP를 한정

결과 외부에 열린 범위를 줄임

결과

항목	전	후
서버	고객사 Windows 서버와 오래된 ASP.NET	AWS EC2와 RDS로 재구축
관리 접근		운영 IP 허용목록만 열고 RDP 차단
업로드 파일	검증 없이 저장	파일과 입력 무결성 검증 뒤 저장
복구		사고 인지부터 복구까지 4일
같은 원인의 재발		재직 중 확인되지 않음

파일과 입력 무결성 검증을 강화한 서버를 다시 배포했습니다. VPN을 쓸 수 없는 구간은 서비스 포트와 출발지 IP를 한정했습니다. AWS로 옮긴 뒤 제가 재직하는 동안 같은 원인의 공격은 다시 확인되지 않았습니다.

트러블슈팅

대형병원 키오스크 CMS의 권한 결함 수정과 보안점검 대응

대형병원 키오스크를 원격으로 관리하는 CMS를 혼자 만들어 운영했고, 세션 검증보다 권한 부여가 먼저 일어나던 결함을 고쳐 병원 보안점검을 통과했습니다.

시기	2023.04–2025.03
환경	TypeScript, Angular, Express, MariaDB, Redis, WebSocket, JWT, bcrypt, RBAC

문제 상황

고객은 4만 평 규모의 대형병원이었습니다. 병원 내부망의 키오스크 약 200대는 장애가 날 때마다 담당자가 현장으로 가야 했습니다. 이 부담을 줄이려고 원격 관리 CMS를 혼자 만들어 운영했습니다. CMS에는 화면 미러링과 원격 명령, 예약 배포를 넣었습니다.

납품하는 CMS는 병원 보안점검을 통과해야 했고, 권한 분리와 계정 정책을 병원 기준에 맞춰야 했습니다.

1. **요청** 관리자나 운영자가 CMS에 로그인해 요청을 보냅니다
2. **세션 검증** Redis에서 세션을 먼저 확인합니다
3. **권한 부여** 검증이 끝난 세션에만 역할 권한을 붙입니다
4. **원격 제어** WebSocket으로 키오스크 화면을 보고 명령을 보냅니다
5. **배포** 예약과 그룹 단위로 배포하고 문제가 생기면 롤백합니다

원인 분석

병원 보안 요구에 맞춰 인증과 권한 구조를 고치던 중에, Redis에서 세션 검증이 끝나기 전에 임시 권한이 먼저 붙는 구간을 찾았습니다. 요청 타이밍에 따라 검증되지 않은 세션이 권한을 쥘 수 있는 구조였습니다.

해결 과정

- 권한 부여를 세션 검증 뒤로 옮겨 처리 순서를 바로잡았습니다.
- 서버 개인키로 서명하는 JWT 인증과 bcrypt 비밀번호 해시, 관리자와 운영자 역할을 나눈 RBAC를 구현했습니다.
- 90일마다 비밀번호를 바꾸게 하고, 로그인 실패 횟수 제한과 재사용 금지, 계정 잠금 정책을 적용했습니다.
- ORM으로 쿼리를 만들어 SQL을 직접 조립하는 곳을 줄였습니다.
- 연동 문제는 브라우저 개발자 도구의 Network 탭에서 엔드포인트와 요청 내용, Authorization 헤더의 JWT를 확인하며 분석했습니다.

세션 검증 전에 임시 권한이 붙는 구간

해결	권한 부여를 세션 검증 뒤로 옮김
결과	검증되지 않은 세션이 권한을 쥐는 구간을 없앰

병원의 계정 정책 요구

해결 정기 변경, 실패 횟수 제한, 재사용 금지, 계정 잠금

결과 납품한 CMS 전체가 병원 보안점검 통과

장애마다 현장으로 가야 하는 운영

해결 WebSocket 화면 미러링과 원격 명령

결과 원격으로 풀 수 있는 장애는 CMS에서 처리

외부 인증기관 접속이 막힌 망분리 환경

해결 NICE CPClient, Electron 앱, IIS와 .NET 로컬 인증 서버를 콜백으로 연결

결과 망분리 안에서 본인인증 흐름 배포

결과

항목	전	후
권한 부여 순서	요청, 임시 권한 부여, 세션 검증	요청, 세션 검증, 권한 부여
계정 정책		정기 변경, 실패 횟수 제한, 재사용 금지, 계정 잠금
키오스크 장애 처리	담당자가 현장으로 이동	원격으로 풀 수 있는 장애는 CMS로 처리

납품한 CMS 전체가 병원 보안점검을 통과했습니다.

프로젝트

ETK-Scanner

비용 상한이 있는 LLM 소스코드 취약점 진단

기간	2026.07
역할	파이프라인 설계, 구현, 검증
기술	Python ast, Claude Haiku, Sonnet, SARIF 2.1.0, GitHub Actions, OSV.dev API
저장소	github.com/ki0915/etk-scanner

소개

정적분석으로 의심 구간을 먼저 좁히고 LLM 검증은 필요한 곳에만 쓰는 취약점 스캐너를 만들어, 실습 백엔드와 오픈소스의 권한 문제를 찾았습니다.

위험 함수 호출만 찾는 규칙 기반 도구는 권한 우회나 검증 우회처럼 위험 함수 없이 생기는 결함을 잡지 못합니다. 모든 함수를 LLM에 보내면 비용 때문에 CI에 붙이기 어렵습니다. 비용 대비 효율이 좋은 취약점 진단 도구를 만들고자 시작했습니다.

처리 흐름

1. **후보 좁히기** 정적분석으로 보안 판단 함수와 위험 지점을 후보로 뽑습니다. LLM 비용이 들지 않습니다
2. **저비용 분류** 저비용 모델이 후보마다 한 번 분류합니다
3. **고비용 검증** 남은 소수 후보만 고비용 모델이 문맥을 여러 번 확인합니다
4. **비용 회계** 호출마다 비용을 기록하고 예산에 닿으면 멈춥니다
5. **보고** SARIF로 GitHub 보안 탭과 CI job 결과에 연결합니다

구현

비용이 적은 단계부터 거르도록 정적분석, 저비용 모델 분류, 고비용 모델 검증 순서로 나눴습니다.

후보에는 eval 같은 위험 함수와 함께 validate나 auth처럼 보안 판단을 하는 함수를 넣었습니다. 권한 우회나 검증 우회는 위험 함수 호출 없이도 생기기 때문입니다.

- Python 코드는 AST로, JavaScript와 TypeScript는 정규식으로 보안 판단 함수를 후보로 뽑습니다. 이 단계는 LLM 비용이 들지 않습니다.
- 저비용 모델이 후보마다 한 번 분류하고, 남은 소수만 고비용 모델이 여러 차례 문맥을 확인하며 검증합니다.
- 모든 LLM 호출의 비용을 기록하고, 정한 예산에 닿으면 분석을 멈춥니다.
- 결과를 SARIF로 내보내 GitHub 보안 탭에 올리고, GitHub Actions에서 PR과 push마다 정적 모드를 돌려 검사가 실패하면 job을 실패로 처리합니다.

트러블슈팅

큰 저장소에서 검사 조각이 불어나 비용이 커짐

해결 테스트와 문서 폴더를 검사 대상에서 제외

결과 검사 조각 수를 줄임

저비용 분류가 음성이라 하면 상위 검증이 막힘

해결 재현율 우선 분류, 응답을 못 읽으면 후보로 유지

결과 놓치던 사례가 다시 검증 단계로 넘어감

재현된 결함과 보고할 만한 결함이 섞임

해결 악용 가능성 확인 단계와 보안 영향 필터

결과 기능 버그를 자동으로 걸러 냄

이미 알려진 취약점을 제보 직전까지 감

해결 OSV와 GHSA 중복 확인 단계 추가

결과 중복 제보를 막음

성과

항목	전	후
위조 토큰 요청	HTTP 200, 보호 데이터 노출	HTTP 401로 차단
SecurityEval 취약 코드		121개 중 111개 탐지

직접 만든 TypeScript 실습 백엔드를 검사하자 소스에 박힌 JWT 서명 키가 잡혔습니다. 이 키를 알면 누구나 토큰을 위조할 수 있습니다. 키를 환경변수로 옮기기 전과 후를 실제 HTTP 서버로 비교했습니다.

공개 SecurityEval 데이터의 취약 코드에도 스캐너를 돌려 몇 개를 찾는지 확인했습니다.

이 스캐너를 활용해 한 오픈소스 프로젝트의 인가 처리 문제를 소스 수준에서 분석했습니다. 재현 자료와 원인 분석을 유지관리자에게 보냈고, 영향 확인과 수정 예정 회신을 받았습니다. 공개 전이라 프로젝트 이름과 세부 내용은 적지 않습니다.

실행 화면

```
$ python -m unittest discover -s tests
.........................................

----------------------------------------------------------------------

Ran 45 tests in 2.152s



OK
```

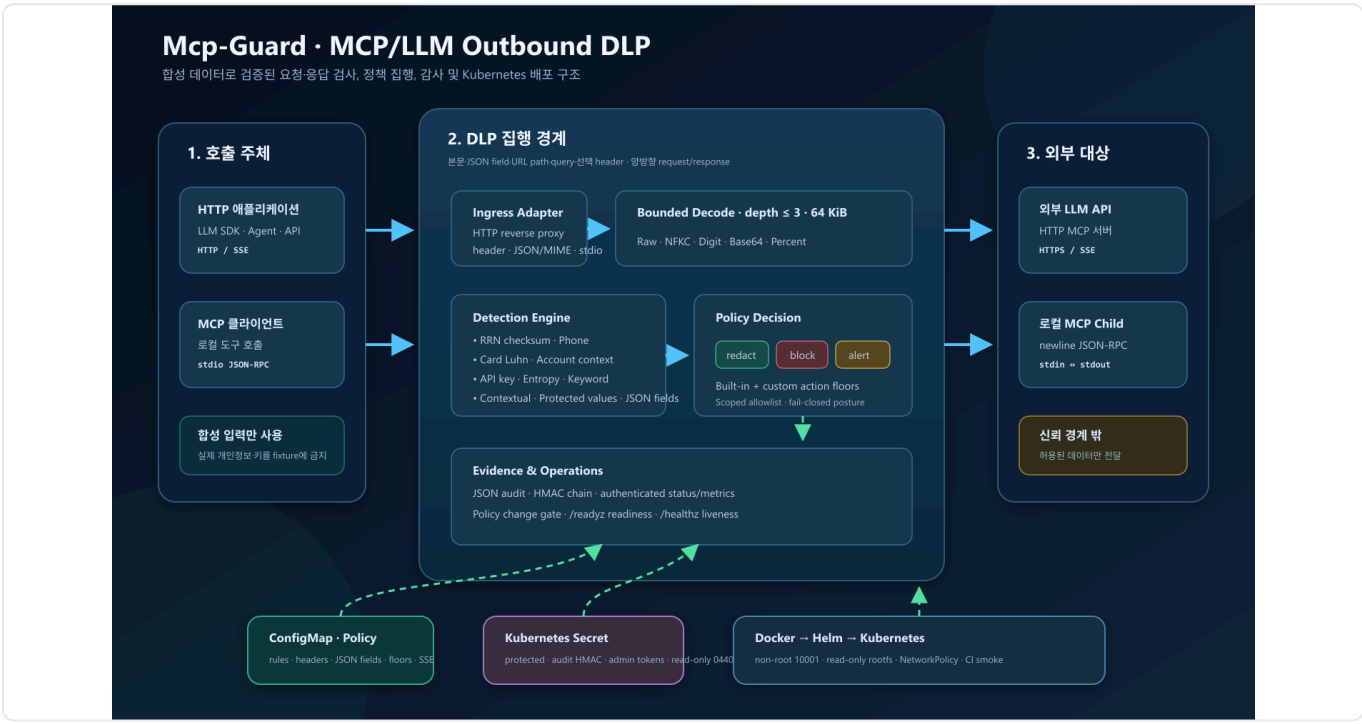
python -m unittest discover -s tests 실행 결과

프로젝트

Mcp-Guard

LLM과 MCP 통신의 개인정보 유출 방지 프록시

기간	2026.07
역할	정책과 탐지 설계, 구현, 검증
기술	Python, FastAPI, uvicorn, httpx, regex, Docker, Helm, k3d, GitHub Actions
저장소	github.com/ki0915/Mcp-Guard



Mcp-Guard 아키텍처

소개

LLM과 MCP 도구로 나가는 요청에서 주민등록번호와 API 키 같은 민감 정보를 찾아 가리거나 막는 프록시를 만들었습니다.

생성형 AI를 업무에 쓰면 사람이 붙여 넣은 문서나 로그에 개인정보와 시크릿이 섞이기 쉽습니다. MCP 서버는 표준 입출력으로 JSON-RPC 메시지를 주고받는 경우가 많습니다. 그래서 네트워크 경계에 둔 HTTP 프록시만으로는 내용을 볼 수 없습니다.

주요 기능

HTTP 리버스 프록시

요청과 응답 본문, URL, SSE 이벤트까지 검사합니다

stdio MCP 어댑터

MCP 서버를 자식 프로세스로 띄워 양방향으로 검사하고, 막으면 JSON-RPC 오류를 돌려줍니다

한국 개인정보 탐지

주민등록번호 검증 숫자, 카드 Luhn, 계좌 은행 문맥으로 확인합니다

보호값 레지스트리

조직이 정한 기밀값을 해시로만 저장해 대조합니다

감사 로그와 지표

원래 값 없이 JSON Lines로 남기고 Prometheus로 보냅니다

처리 흐름

1. **받기** HTTP 리버스 프록시와 stdio 어댑터가 요청과 응답을 받습니다
2. **풀어 보기** NFKC 정규화, 숫자 사이 공백과 보이지 않는 문자 제거, 크기를 제한한 Base64 디코딩
3. **찾기** 주민등록번호, 카드, 계좌, 시크릿, 조직 기밀 탐지기가 위치를 찾습니다
4. **판정** 정책에 따라 가리기, 막기, 경보 가운데 하나를 고르고, 판단이 어려우면 막습니다
5. **기록** 원래 값을 뺀 감사 로그와 Prometheus 지표를 남깁니다

구현

HTTP 프록시와 stdio 어댑터에 같은 정책을 적용해 두 경로의 판단이 같게 했습니다. 검사는 원문 위치를 유지한 파생 문자열로 하고, 마스킹은 원래 입력의 해당 범위에 적용했습니다. 판단이 어려우면 막고, 감사 로그에는 원래 값을 남기지 않게 했습니다.

- HTTP 리버스 프록시는 요청과 응답 본문, URL, SSE 이벤트를 검사하고, stdio 어댑터는 MCP의 줄 단위 JSON-RPC를 같은 정책으로 검사합니다.
- 주민등록번호는 날짜와 검증 숫자로, 카드번호는 IIN과 Luhn 검사로, 계좌번호는 앞뒤의 은행 문맥으로 확인합니다.
- 유니코드 NFKC 정규화, 숫자 사이 공백과 보이지 않는 문자 제거, 크기를 제한한 Base64 디코딩으로 우회 표현을 풀어 검사합니다.
- 정책 스키마 검증과 시크릿 분리, 만료 시간이 있는 허용 예외, Prometheus 지표, Dockerfile과 Helm 차트를 만들었습니다.

트러블슈팅

처음 버전은 원문 그대로만 검사했고, 띄어 쓴 주민등록번호나 전각 숫자, Base64로 감싼 값이 그대로 통과했습니다. 우회 기법을 넣은 합성 데이터로 그 차이를 쟀습니다.

HTTP만 보는 프록시는 stdio MCP 서버에 닿지 못함

해결 줄 단위 JSON-RPC를 검사하는 stdio 어댑터

결과 두 통신 경로에 같은 정책을 적용

띄어 쓴 번호, 전각 숫자, Base64로 감싼 값이 통과

해결 원문 위치를 유지하는 정규화 뒤 검사

결과 합성 우회 사례 탐지가 크게 늘어남

실제 기밀값이 설정 이력에 남을 위험

해결 기밀값은 Kubernetes Secret에만 두고 해시로 대조

결과 ConfigMap과 Helm 이력에 원래 값이 남지 않음

원문을 그대로 보내는 경보 설정으로 정책이 약해질 위험

해결 주민등록번호, 카드, 시크릿은 경보로 낮출 수 없게 함

결과 고위험 값은 항상 가리거나 막음

성과

항목	전	후
민감 우회 사례 탐지	60건 중 10건	60건 중 56건
정상 사례 오탐	30건 중 0건	30건 중 0건
HTTP 프록시 추가 지연		평균 3.10ms

우회 기법을 넣은 합성 데이터로 정규화 전후의 탐지 수와 오탐을 비교했습니다. 합성 데이터는 민감 사례와 정상 사례로 나눴습니다. 프록시를 거칠 때 늘어나는 지연은 로컬에서 같은 요청을 짝지어 비교했습니다.

탐지와 정책 기준, 정규화 방식, 구현 검토와 최종 반영은 직접 판단했습니다.

실행 화면

```
$ python -m pytest -q
.................................................................... [ 21%]
..................s................................................. [ 42%]
.................................................................... [ 64%]
.................................................................... [ 85%]
.........................................                           [100%]
334 passed, 1 skipped in 3.26s
```

python -m pytest -q 실행 결과

프로젝트

EvidScope

AI 에이전트의 위험 행동을 판정 모델로 가려내 감사 증거로 남기는 보안 도구

기간	2026
역할	문제 정의, 우선순위와 모델, 비용 조건 결정, 최종 검토
기술	JEV-9B, Qwen3-4B, Python, PyTorch, Transformers, PEFT, QLoRA (4비트), Node.js, SQLite, Ed25519
저장소	github.com/ki0915/EvidScope

소개

Claude Code와 Codex가 실행하는 도구 호출을 실시간으로 받아 규칙과 직접 학습한 판정 모델로 위험한 행동을 가려내고, 당시 권한과 정책, 사람의 판단과 함께 서명 원장에 남겨 감사 증거로 쓰게 하는 도구입니다.

기업이 AI를 쓰면 실행 로그만으로는 AI가 어떤 권한과 근거로 판단했는지 설명하기 어렵고, 원본 증거를 확보해 책임추적성을 부여하기도 어렵습니다. 이런 상황에서 AI 활동 가시성과 감사 거버넌스를 지원하는 보안 도구를 만들었습니다.

처리 흐름

1. **규칙** 위험 규칙 67개와 행동 순서 규칙이 먼저 경보를 냅니다
2. **JEV-9B** 모든 세션의 위반 여부를 확률과 함께 판정하고, 애매하면 보류합니다
3. **Qwen3-4B** 위반이나 보류로 본 세션만 다시 받아 증거 행동과 위험 유형을 짚습니다
4. **서명된 대응표** 모델이 아니라 대응표가 인공지능 기본법, ISMS-P, EU AI Act 항목을 계산합니다
5. **서명 원장** 당시 권한과 정책, 사람의 판단과 함께 남겨 감사 증거로 씁니다

구현

판정 파이프라인을 규칙, 1차 판정 모델, 2차 판정 모델, 서명된 대응표의 네 단계로 나눴습니다. 위험 규칙 67개와 행동 순서 규칙이 먼저 경보를 내고, JEV-9B가 모든 세션의 위반 여부를 확률과 함께 판정합니다. JEV-9B가 위반이나 보류로 본 세션과 규칙 경보를 정상으로 본 세션만 Qwen3-4B가 다시 받아 증거가 된 행동과 위험 유형을 짚습니다. 법 조항은 모델이 쓰지 않고 서명된 대응표가 계산합니다.

판정이 흔들리지 않도록 모델은 규칙 경보를 지울 수 없게 했고, 경보를 낮추려면 두 모델이 모두 동의해야 하며, 보안 기능 해제나 기록 변조 같은 보호 경보는 낮추지 못하게 했습니다.

- JEV-9B는 Qwen3.5-9B 위에 판정 블록을 얹은 모델로, 글을 생성하지 않고 예와 아니오를 확률과 함께 한 번에 냅니다. 확률로 판단을 보류할 수 있어서 모든 세션의 1차 판정을 맡겼습니다.
- Qwen3-4B는 규칙 소견이 없는 세션에서도 어느 행동이 증거인지 짚어 내서, 1차 판정에서 걸린 세션의 근거 행동과 위험 유형을 맡겼습니다.
- 두 모델은 같은 학습 데이터로 4비트 QLoRA 학습을 했고, 보류 문턱은 시험과 같은 분포의 검증 분할에서만 골랐습니다.
- 공개 데이터는 ASSEBench를 학습과 평가에 쓰고, R-Judge와 ATBench는 외부 시험으로만 남겼습니다. 외부 시험과 겹치는 공개 기록은 학습 데이터에서 기계적으로 뺐습니다.

- 모든 라벨 세션은 프롬프트 없는 한 레코드 형식으로 모아, 판정 모델을 바꿔도 같은 데이터와 채점기로 학습하고 비교하게 했습니다.
- Claude Code 실행 전 hook에서는 감사 설정 변경, 보안 기능 끄기, 비밀을 읽어 바로 외부로 보내는 명령만 막고 막은 사실을 원장에 남깁니다.
- 한국 인공지능 기본법 제30~36조와 제40조의 의무를 요구사항 18개로 정리하고, 경보를 ISMS-P와 EU AI Act 항목까지 서명된 대응표로 연결했습니다.

트러블슈팅

모델이 경보 이름만 보고 답하는 지름길

해결 같은 경보에 정답이 다른 대조 쌍, 경보 이름을 가린 변형, 규칙 출력을 지운 변형을 섞은 합성 데이터

결과 경보 이름만 보고 답하는 지름길을 막음

학습과 평가 사이의 누수

해결 외부 시험과 겹치는 공개 기록을 기계적으로 빼고 누수 검사 4종을 둠

결과 하나라도 걸리면 학습 세트를 만들지 않음

요청마다 서명 원장 전체를 다시 검증하던 병목

해결 마지막 검증 지점 이후 변경분만 검증하고, 변조가 감지되면 처리를 멈춤

결과 합성 부하 시험의 접수가 600건 중 84건에서 600건 모두로

성과

항목	전	후
학습에 쓰지 않은 위협 계열 시험 1,050건	규칙만으로 정확도 약 70%	학습한 두 판정 모델 모두 정답, 놓친 위협 없음
위협 영역 교차 검증		JEV-9B 평균 정확도 97.0%
공개 데이터 ASSEBench 시험		JEV-9B 96.9%
외부 시험 R-Judge	63%	87% (학습 회차를 거듭하며)
원장 검증 합성 부하 시험	600건 중 84건 접수	600건 모두 처리

트러블슈팅

서버 7대 통합과 여러 겹의 백업 설계

노후 서버 7대를 Hyper-V로 합치며 외부에서 바로 들어오던 원격 접속을 없애고, 스냅샷 하나에 기대던 복구를 여러 겹의 백업으로 바꿨습니다.

시기	2023.04–2025.03
환경	Windows Server 2022, Hyper-V, AWS, MSSQL 복제, IIS

문제 상황

사내 서버 7대는 오래된 장비였습니다. 웹 서버와 DB에는 외부에서 바로 원격 접속할 수 있었고, 출결 서버는 Windows XP 기반이었습니다. 장애가 나면 기댈 수 있는 것은 Hyper-V 스냅샷 하나였고, 담당자가 없을 때도 복구할 수 있는 절차가 필요했습니다.

1. **점검** 서버마다 역할과 외부 노출 상태를 확인했습니다
2. **통합** Windows Server 2022와 Hyper-V로 합쳐 옮겼습니다
3. **접근 정리** 인바운드는 기본 차단하고 원격 접속은 내부망 경로로만 열었습니다
4. **복구 수단 나누기** 변경 제한 백업, AWS 서버 이미지, MSSQL 복제, 이중화 서버
5. **문서화** 담당자가 없어도 따라 할 수 있게 복구 절차를 남겼습니다

원인 분석

서버마다 역할과 외부 노출 상태를 하나씩 점검하면서 알았습니다.

해결 과정

- 서버를 Windows Server 2022와 Hyper-V 환경으로 통합하고 이관했습니다.
- 방화벽은 인바운드를 기본 차단하고 아웃바운드는 필요한 것만 허용하게 다시 짰고, 웹 서버와 DB에 외부에서 바로 들어오던 원격 접속을 없앴습니다.
- 복구 수단을 별도 볼륨에 쓰기를 제한한 백업, AWS 서버 이미지, MSSQL 복제, API가 실패하면 이중화 서버로 넘기는 흐름으로 나눴습니다.
- 백업 접근은 사내 게이트웨이 IP 중심으로 제한하고 복구 절차를 문서로 남겼습니다.
- Windows XP 기반 출결 서버를 새로 개발해 IIS에 올리고 인트라넷 DB와 연동했습니다.

웹 서버와 DB에 외부에서 바로 원격 접속

해결	방화벽 재설계와 내부망 경로
결과	외부 원격 접속 경로를 없앰

Hyper-V 스냅샷 하나에 기대던 복구

해결 별도 볼륨 백업, AWS 이미지, MSSQL 복제

결과 복구 수단을 여러 겹으로 나눔

결과

항목	전	후
복구 수단	Hyper-V 스냅샷 하나	변경 제한 백업, AWS 서버 이미지, MSSQL 복제
외부 원격 접속	웹 서버와 DB에 바로 접속	내부망 경로로만 접근
API 장애	이중화 없음	실패하면 이중화 서버로 전환

트러블슈팅

5성급 호텔 서버의 디스크 고갈 장애와 재발 방지

배포 때마다 쌓인 Docker 이미지와 중지된 컨테이너가 디스크를 채워 서비스가 멈춘 장애를 복구하고, 주기 정리를 자동화해 같은 원인의 장애를 막았습니다.

시기 2023.04–2025.03

환경 Docker, Ubuntu, cron, Grafana, Prometheus

문제 상황

5성급 호텔 고객사의 서버는 Ubuntu 위에서 Docker 컨테이너로 서비스를 돌렸습니다. 관리자 페이지와 DB, 서버 상태는 SSL VPN으로 접속해 점검했습니다. Grafana와 Prometheus 지표가 임계치에 가까워지면 미리 조치해 왔습니다.

1. **장애 확인** 서비스가 멈춰 디스크가 가득 찬 것을 확인했습니다
2. **원인 찾기** 배포마다 남은 이미지와 중지된 컨테이너를 찾았습니다
3. **복구** 쓰지 않는 이미지와 컨테이너를 지우고 서비스를 다시 올렸습니다
4. **자동화** 같은 정리를 cron으로 주기 실행했습니다
5. **재현과 가드** 개인 랩에서 장애를 재현하고 단계적 정리 가드를 만들었습니다

원인 분석

서비스가 멈춰 확인해 보니 서버 디스크가 가득 차 있었습니다. 배포 때마다 남은 미사용 Docker 이미지와 중지된 컨테이너가 원인이었습니다.

해결 과정

- 중지된 컨테이너와 쓰지 않는 이미지를 지워 공간을 확보하고 서비스를 다시 올렸습니다.
- 같은 정리 작업을 cron에 등록해 사람이 챙기지 않아도 주기적으로 돌게 했습니다.
- 로그와 Docker 이미지 정리 절차, 서버 점검 표준을 문서로 남겼습니다.
- 같은 장애를 개인 랩에서 작은 디스크로 재현해, 롤백용 이미지는 지키면서 디스크 사용률에 따라 단계적으로 정리하는 가드를 다시 만들었습니다.

배포마다 쌓이는 Docker 이미지

해결 cron 주기 정리

결과 같은 원인의 서비스 중단이 다시 없었음

롤백용 이미지까지 지울 위험

해결 디스크 사용률에 따라 단계적으로 정리하는 가드

결과 재현 실험의 배포가 모두 성공

결과

항목	전	후
디스크 정리		cron으로 주기 실행
재현 실험의 배포	14회 중 7회 실패	14회 모두 성공

cron 주기 정리를 건 뒤로는 같은 원인으로 디스크가 차서 서비스가 멈추지 않았습니다. 개인 랩의 재현 실험에서는 정리 없이 배포하면 절반이 실패했고, 가드를 적용하자 모두 성공했습니다.

기술 문서

Docker 컨테이너 보안 가이드 저술

기간	2026년 개인 저술
역할	작성자
저장소	github.com/ki0915/Docker-Security-Guide

소개

위험도와 구현 비용을 기준으로 Docker 컨테이너 보안 대응을 다시 정리한 2026년 개인 기술 문서입니다.

OWASP Docker Top 10, CIS Docker Benchmark, NIST SP 800-190을 참고해 D01부터 D10까지의 대응 방법을 위험도와 구현 비용에 따라 재구성했습니다. 항목마다 설정 코드 예와 설명을 붙였습니다.

주요 기능

이미지 관리

이미지를 최소화하고 digest로 고정하며, CI에서 Trivy로 검사하는 방법을 설명합니다.

실행 권한 제한

non-root 실행과 capability 제거, rootless 구성을 코드 예와 함께 다룹니다.

빌드 시크릿과 네트워크

BuildKit 시크릿으로 빌드 중 비밀값을 다루는 방법과 네트워크 격리를 정리합니다.

런타임 관찰

cAdvisor와 Falco로 실행 중인 컨테이너를 관찰하는 방법을 설명합니다.

프로젝트

TryHackMe 모의 침투 실습과 보고서

소개

허가된 워게임 환경에서 모의 침투를 연습하고, 공격 경로와 개선 방안을 보고서로 정리한 학습 기록입니다.

TryHackMe의 Jr Penetration Tester 경로와 웹 해킹 모듈을 실습했습니다. 정찰부터 침투, 권한 상승, 개선 권고까지를 OSCP 보고서 양식으로 작성하며 보고서 쓰는 연습을 했습니다.

주요 기능

서비스 열거와 연계 분석

Nmap으로 서비스를 열거하고, SMB 익명 접근과 IIS 공유 디렉터리 설정 오류가 이어지는 경로를 분석했습니다. 권한 상승 경로까지 실습해 보고서로 남겼습니다.

웹 취약점 실습

SQL 인젝션, XSS, IDOR 등 웹 해킹 모듈을 Burp Suite로 실습했습니다.

Active Directory 학습

Active Directory 구성과 Kerberos, NetNTLM 인증 방식을 학습해 정리했습니다.

개인 학습

WIKI

기간	2023년 개인 프로젝트
역할	프론트엔드·백엔드 개인 개발
기술	TypeScript
저장소	github.com/ki0915/wiki-backend

소개

게시판과 권한별 접근 제한, 실시간 채팅을 갖춘 위키 서비스를 프론트엔드부터 백엔드까지 직접 만든 2023년 개인 프로젝트입니다.

TypeScript와 Express로 API 서버를, Angular로 클라이언트를 나눠 만들었습니다. 게시글 CRUD와 파일 업로드, JWT 인증과 bcrypt 비밀번호 해시, 역할별 권한을 구현했고, Socket.IO 채팅을 직접 붙였습니다.

주요 기능

실시간 채팅

Socket.IO로 받은 메시지를 Chat 모델에 저장하고, 방마다 receiveMessage 이벤트로 참여자에게 전달합니다.

게시글과 파일 업로드

게시글 CRUD API와 파일 업로드를 제공하며, 긴 게시글이 저장되도록 저장 형식을 STRING에서 TEXT로 바꿨습니다.

권한별 접근 제한

역할 등급에 따라 기능을 나눠, 게시글 삭제는 권한 3 이상 사용자만 할 수 있게 했습니다.

요청 보호와 운영 구성

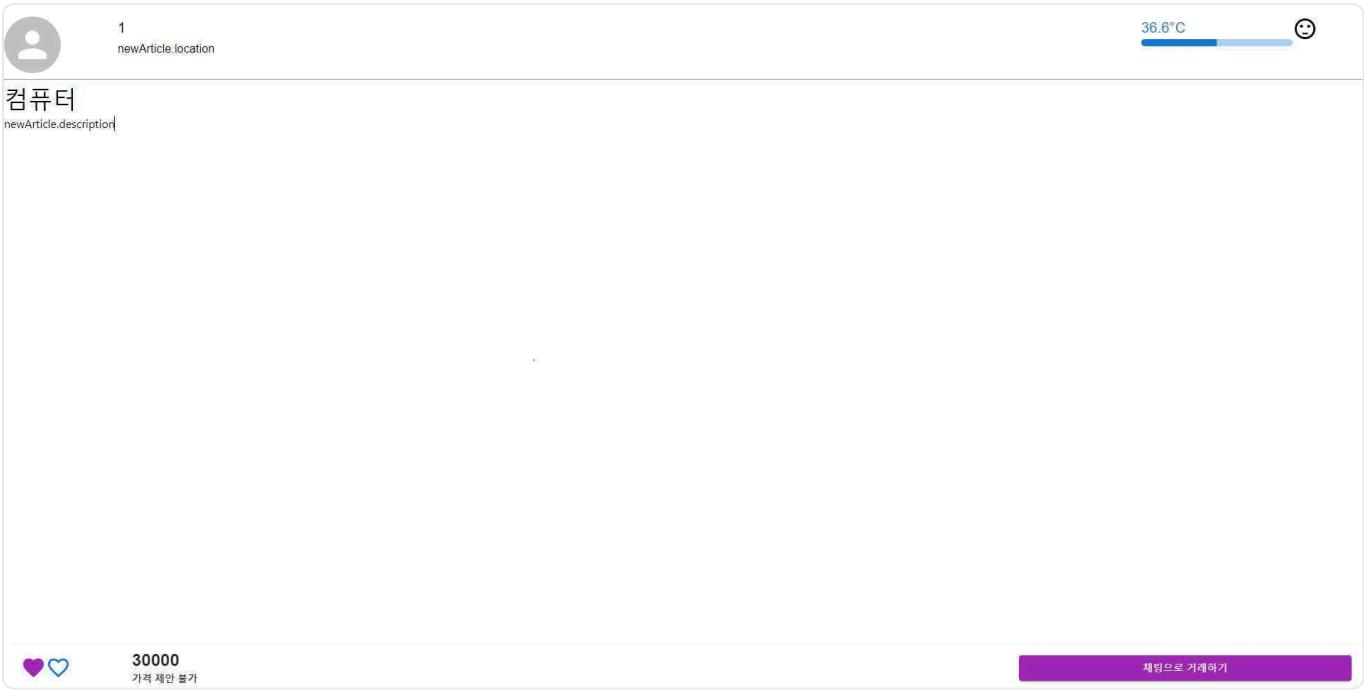
helmet과 hpp 미들웨어를 적용하고, Docker Compose와 PM2, Winston으로 배포와 로그를 구성했습니다.

개인 학습 프로젝트

Carrot 클론

기간 2022년

역할 React 프론트엔드·Node.js 백엔드 개발



거래 화면

소개

상품을 올리고 둘러보는 거래 서비스를 본떠 화면과 서버를 연결해 본 2022년 학습 프로젝트입니다.

화면은 React와 Axios, Material UI로, 서버는 Express와 MySQL, Sequelize 기반 MVC 구조로 만들었습니다. 두 부분을 연결해 상품 조회, 상세 보기, 등록 기능을 구현했습니다. 채팅 화면은 더미 데이터로 화면 구성만 만들었습니다.

주요 기능

상품 조회와 등록

서버 API에서 상품 목록과 상세 정보를 받아 보여 주고, 사용자가 입력한 상품을 등록합니다.

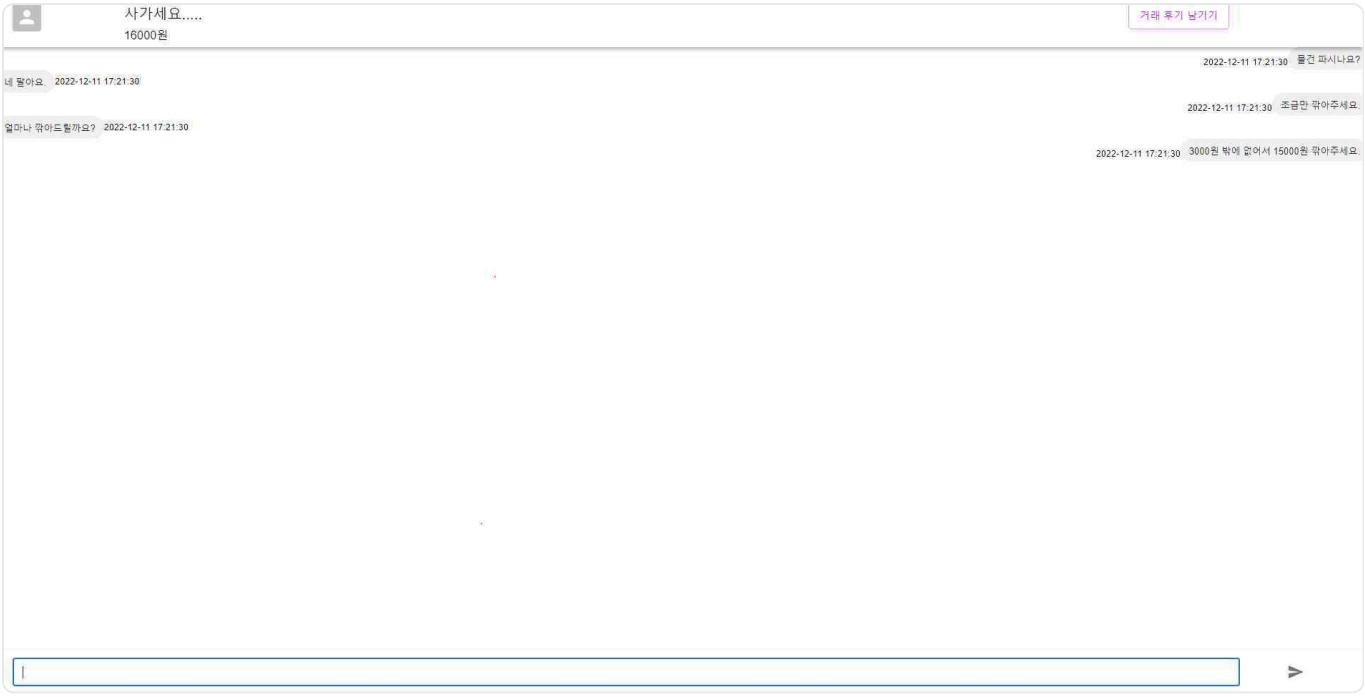
거래 화면

트러블슈팅

API 연동 문제 해결

해결 CORS와 JSON 파싱 문제를 API 요청 도구로 확인해 고쳤고, useEffect 의존성을 조정해 요청이 반복되던 문제를 해결했습니다.

화면


사가세요.....
16000원
거래 후기 남기기
2022-12-11 17:21:30
물건 파시나요?
네 팔아요. 2022-12-11 17:21:30
2022-12-11 17:21:30
조금만 깎아주세요.
얼마나 깎아드릴까요? 2022-12-11 17:21:30
2022-12-11 17:21:30
3000원 밖에 없어서 15000원 깎아주세요.

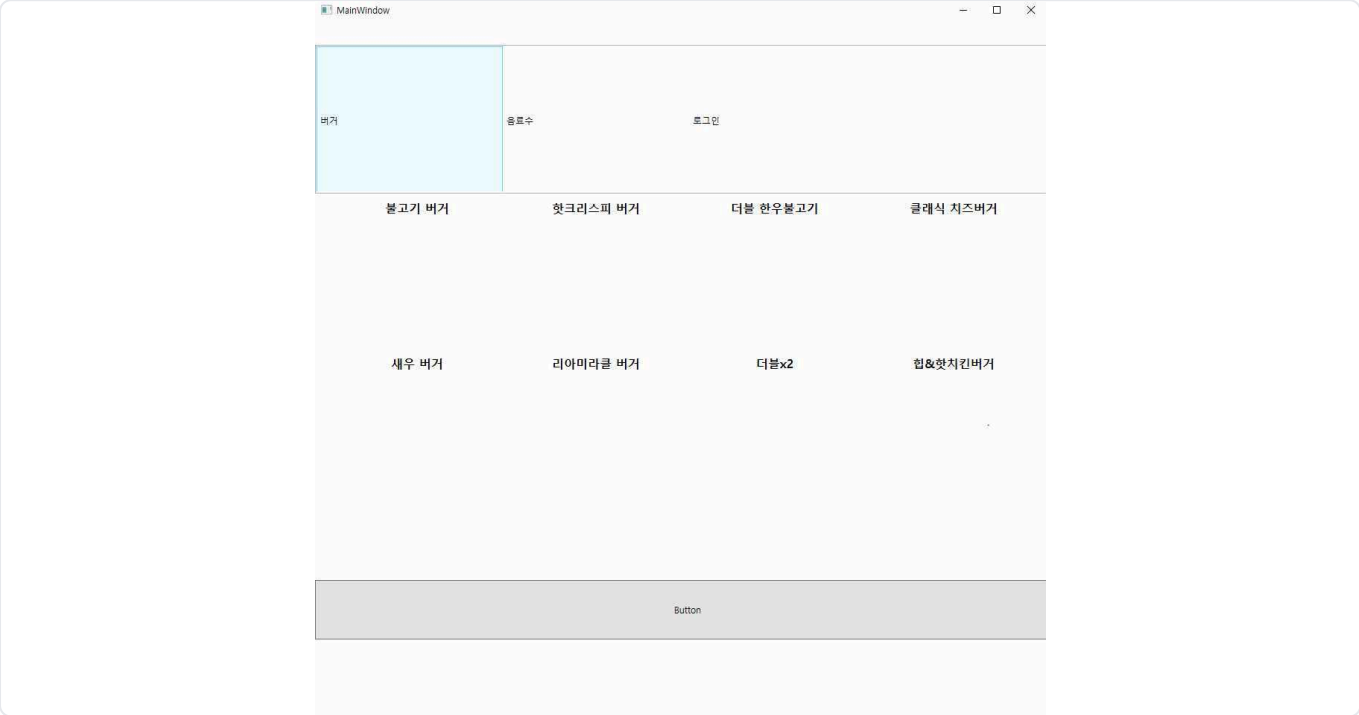
채팅 화면

학습용 개인 프로젝트

Kiosk

기간 2021.09–2022.01

역할 개인 개발



주문 화면

소개

2021년 9월부터 2022년 1월까지 주문 키오스크의 상품 선택부터 주문 전송까지 만들어 본 학습용 개인 프로젝트입니다.

C# WPF로 주문 화면을, Node.js와 Express, MSSQL로 서버를 만들었습니다. 상품 목록과 장바구니, 관리자용 상품 관리를 구현하고, 주문 금액은 서버로 보내 검증하게 했습니다. 회사에서 개발한 병원 키오스크 CMS와는 별개의 프로젝트입니다.

주요 기능

주문 화면

상품 목록에서 고른 상품을 장바구니에 담고 주문 금액을 서버로 보냅니다.

주문 화면과 상품 선택 화면

서버 검증과 관리자 기능

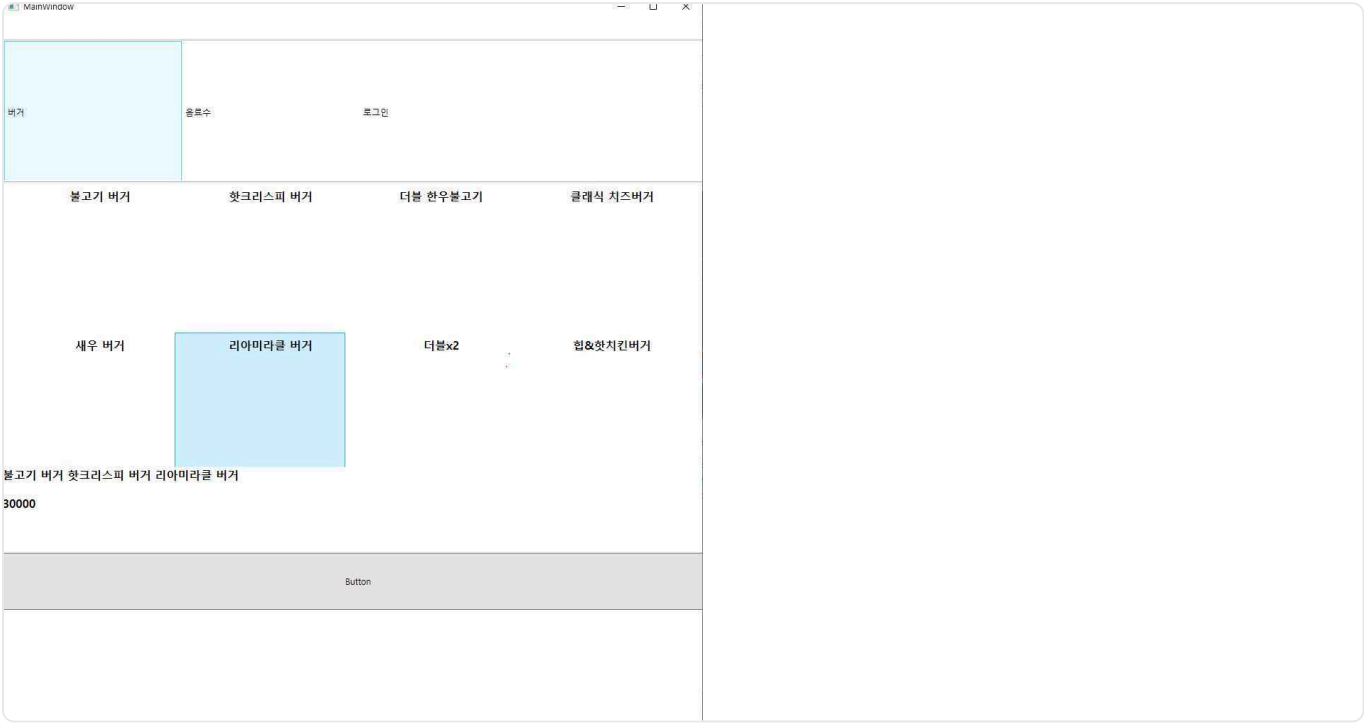
서버가 받은 주문 금액을 검증하고, 관리자는 상품을 등록하고 고치고 지울 수 있습니다.

트러블슈팅

통신 문제 해결

해결 SQL Server의 TCP/IP 설정과 HTTP 요청의 JSON Content-Type 불일치를 점검해 클라이언트와 서버 사이의 통신 문제를 해결했습니다.

화면



상품 선택